Database Management System Project

on

Virtual Hackathon Tracker

submitted to NIIT University

by

Jayant Pachori
Enrolment No.: BT23GCS185
Email ID: jayant.pachori23@st.niituniversity.in
Section: B3

Course In-Charge: Dr. Anand Kumar Mishra
Assistant Professor, CSE

Computer Science and Engineering

NIIT University, Neemrana, Rajasthan

Academic Year: 2024–25
Semester: Sem II (Jan–June)

Reflections on DBMS Learning

Jayant Pachori

Enrolment No.: BT23GCS185

- The DBMS course helped me understand how databases work and why they are important. Before this course, I didn't know much about how data is stored or used. But now, I know how to create and manage a database and how to get useful information from it.

- In this project, Virtual Hackathon Tracker, I used everything I learned. I made an ER diagram, created tables, and wrote SQL queries. This helped me understand how to connect data and get the right output. It was my first time working on a real database project, and it really improved my skills.

- This project also made me feel more confident. Earlier, I was not sure how to write SQL queries, but now I can write them easily. I know these skills will help me in future studies and also in jobs. Most tech companies use databases, so learning DBMS is very useful.

- Databases are used in almost every field in tech. They help in storing, managing, and using data properly. Knowing how to use them is an important skill.

- In the future, I want to learn more about databases, like how to use them in websites or apps. This course gave me a good starting point, and I want to keep learning more.

  Overall, I enjoyed this course. It helped me learn something new and useful, and I feel happy with what I have achieved.

Contents

1 Introduction **3**

2 Project Objective **3**

3 ER Diagram **3**

4 Relational Schema **8**

5 SQL Implementation and its Snapshots **9**

6 Sample Queries and Outputs along with snapshots **12**

7 Challenges Faced **18**

8 Conclusion **18**

9 Project's Learning Outcome **19**

10 References **19**

1. Introduction

This project is about creating a Virtual Hackathon Tracker that uses a database to store and manage all important data. It helps in keeping track of participants, teams, events, and scores during a virtual hackathon. The system is built using structured database design and SQL to make data handling easy, fast, and reliable.

2. Project Objective

The objective of this project is to: "*Build a simple and efficient system to manage virtual hackathon details such as participant information, team registrations, event timings, score entries, and final results, using a database to keep all the data well-organized and easy to access.*"

3. ER Diagram

The Entity-Relationship (ER) model designed for the **Virtual Hackathon Tracker** aims to simplify and organize how hackathons are managed, how teams participate, and how submissions are evaluated. Below is a detailed breakdown of the entities, their attributes, relationships, and the design logic behind the schema.

1. Entity Sets and Attribute Explanation

- **Team**
  Represents each participating team in the hackathon system. The attributes are:
  - `team_id` – Unique identifier for each team **(Primary Key).**
  - `team_name` – The official name of the team.
  - `leader_name` – Full name of the team leader.
  - `email` – Email address for the team (used for communication).
  - `contact_number` – Contact number of the team leader.

- **Participant_details**
  Contains data for individual team members. Each participant is associated with one team.
  - `participant_id` – Unique identifier for each participant **(Primary Key).**
  - `team_id` – References the team to which the participant belongs (Foreign Key).
  - `name` – Full name of the participant.
  - `email` – Participant's email address.
  - `role` – Role within the team (e.g., Developer, Designer, Project Manager).
- **Hackathon_details**
  Stores information about each hackathon event.
  - `hackathon_id` – Unique identifier for each hackathon (**(Primary Key)**.
  - `Fullname` – Name of the hackathon (e.g., Hack the Future 2025).
  - `start_date` – Date when the hackathon starts.
  - `end_date` – Date when the hackathon ends.
  - `submission_deadline` – Last date to submit projects.
- **Submission**
  Contains information about the projects submitted by teams.
  - `submission_id` – Unique ID for each project submission **(Primary Key)**.
  - `team_id` – Team that made the submission (Foreign Key).
  - `hackathon_id` – Hackathon to which the project was submitted (Foreign Key).
  - `project_title` – Title of the submitted project.
  - `submission_link` – URL/link to access the project or code repository.
  - `submitted_on` – Date when the project was submitted.
- **Judge**
  Details of individuals who evaluate submissions.
  - `judge_id` – Unique ID for each judge **(Primary Key)**.
  - `Fullname` – Full name of the judge.
  - `email` – Email address of the judge.
  - `expertise` – Domain or field of expertise (e.g., AI, Web Development).

- **Evaluation**
  Stores feedback and scores provided by judges for each submission.
  - `evaluation_id` – Unique ID for each evaluation **(Primary Key).**
  - `judge_id` – Judge who reviewed the submission (Foreign Key).
  - `submission_id` – Project being evaluated (Foreign Key).
  - `score` – Numeric score assigned (e.g., on a scale of 0–100).
  - `remarks` – Optional feedback or comments from the judge.
- **Prize**
  Tracks which teams receive awards for a particular hackathon.
  - `prize_id` – Unique ID for each prize entry **(Primary Key).**
  - `hackathon_id` – Hackathon for which the prize is awarded (Foreign Key).
  - `rank_` – Rank or position secured by the team (e.g., 1 for First Prize).
  - `team_id` – Team that won the prize (Foreign Key).
  - `amount` – Prize money or reward amount (in INR or USD).

2. Explanation of Relationship Sets and Their Connections

- **Team and Participant_details**
  There is a **one-to-many relationship** between teams and participants. Every team consists of one or more participants, while each participant belongs to only one team. This linkage is established through the `team_id` foreign key in the `Participant_details` table. This helps maintain team structure and identity.
- **Team and Submission**
  A team can submit multiple projects (especially if participating in different hackathons), but each submission is tied to a single team. The `Submission` table holds a foreign key `team_id` to define this relationship. This enables tracking of submissions at a team level.
- **Hackathon and Submission**
  A hackathon can receive submissions from multiple teams. Each submission belongs to only one hackathon. The connection is established using `hackathon_id` in the `Submission` table, which allows filtering submissions per event.
- **Submission, Evaluation, and Judge**
  The evaluation process links submissions to judges. Since a judge can evaluate multiple submissions and each submission may be evaluated by multiple judges,

this forms a **many-to-many relationship**. To model this, the `Evaluation` table is introduced as an associative entity containing both `submission_id` and `judge_id` as foreign keys, along with evaluation details like `score` and `remarks`.

- **Team, Prize, and Hackathon**
  The `Prize` table models the award outcomes of each hackathon. It connects a specific team to the prize they won in a particular hackathon. The relationship is **many-to-one** from both the team and hackathon side. This allows us to keep track of awarded teams and their rankings.

3. Candidate Keys and Primary Keys

- `Team`: The primary key is `team_id`. While team names may be unique, the ID is a more reliable way to track teams.
- `Participant_details`: The unique identifier is `participant_id`, which serves as the primary key. Participant names and emails may not be globally unique, so an ID-based key is preferred.
- `Hackathon_details`: The `hackathon_id` acts as the primary key. Each hackathon can be uniquely tracked using this ID.
- `Submission`: The `submission_id` is the primary key here. Every project submission is assigned a unique ID to distinguish it, even if titles are similar.
- `Judge`: Each judge is assigned a unique `judge_id` for identity management. This avoids duplication based on names.
- `Evaluation`: The primary key is `evaluation_id`. Alternatively, a composite key of `judge_id` and `submission_id` could also be used to ensure uniqueness per evaluation.
- `Prize`: The `prize_id` is the primary key. Although rank and hackathon-team combination is usually unique, the ID ensures extensibility for future prize categories.

4. Foreign Key Rationale

- `team_id` in `Participant_details`, `Submission`, and `Prize` tables ensures participant-team, submission-team, and prize-team associations respectively.
- `hackathon_id` in `Submission` and `Prize` connects submissions and awarded teams to their corresponding hackathon events.

- `judge_id` and `submission_id` in `Evaluation` table establish the connection for evaluating which judge has assessed which submission.

Foreign keys help preserve referential integrity by preventing records and ensuring consistent relationships between dependent entities.

5. Structural Constraints: Cardinality and Participation

- **Team – Participant**: One-to-many; total participation from Participant side (every participant must belong to a team).
- **Team – Submission**: One-to-many; optional participation (not all teams may submit).
- **Hackathon – Submission**: One-to-many; optional participation (a hackathon may not always have submissions if it's upcoming).
- **Submission – Evaluation**: Many-to-many; partial participation (some submissions may not be evaluated yet).
- **Judge – Evaluation**: One-to-many; optional participation (a judge may be present but not assigned yet).
- **Hackathon – Prize – Team**: One-to-one per rank; optional participation (only winning teams are recorded).

6. Assumptions Made During Schema Design

- Every team has at least one participant, but a participant cannot belong to multiple teams.
- A team can make only one submission per hackathon to avoid duplication.
- Hackathon events are considered unique and have defined start and end dates with a submission deadline.
- A judge may or may not evaluate submissions, and a submission can have multiple evaluations to ensure fairness.
- Prize rankings (1st, 2nd, 3rd) are unique per hackathon. In case of ties, rankings may be adjusted manually.
- Contact numbers and emails are assumed valid but not treated as unique constraints for flexibility.

- Names (like Fullname, Leader_name, etc.) are not used as keys since they are not guaranteed to be unique.
- Remarks in evaluation can be nullable or optional depending on the evaluation process.
- Future versions may support teams participating in multiple hackathons concurrently and support multi-level judging panels.

4. Relational Schema

The relational schema consists of:

- **Team**(*team_id*, team_name, leader_name, email, contact_number)
- **Participant**(*participant_id*, team_id, name, email, role)
- **Hackathon**(*hackathon_id*, name, start_date, end_date, submission_deadline)
- **Submission**(*submission_id*, team_id, hackathon_id, project_title, description, submission_link, submitted_on)
- **Judge**(*judge_id*, name, email, expertise)
- **Evaluation**(*evaluation_id*, judge_id, submission_id, score, remarks)
- **Prize**(*prize_id*, hackathon_id, rank, team_id, amount)

5. SQL Implementation and its Snapshots

```sql
create database hackathon;
```

Figure 1: Creating the Database

```sql
CREATE TABLE Evaluation (
    evaluation_id INT PRIMARY KEY AUTO_INCREMENT,
    judge_id INT,
    submission_id INT,
    score DECIMAL(4,2),
    remarks varchar(40),
    FOREIGN KEY (judge_id) REFERENCES Judge(judge_id),
    FOREIGN KEY (submission_id) REFERENCES Submission(submission_id)
);
```

Figure 2: Creating the Evaluation Table

```sql
CREATE TABLE Hackathon_details (
    hackathon_id INT PRIMARY KEY AUTO_INCREMENT,
    Fullname VARCHAR(100),
    start_date DATE,
    end_date DATE,
    submission_deadline DATE
);
```

Figure 3: Creating the Hackathon_details Table

```sql
CREATE TABLE Judge (
    judge_id INT PRIMARY KEY AUTO_INCREMENT,
    Fullname VARCHAR(100),
    email VARCHAR(100),
    expertise VARCHAR(100)
);
```

Figure 4: Creating the Judge Table

```sql
CREATE TABLE Participant_details (
    participant_id INT PRIMARY KEY AUTO_INCREMENT,
    team_id INT,
    name VARCHAR(100),
    email VARCHAR(100),
    role VARCHAR(50),
    FOREIGN KEY (team_id) REFERENCES Team(team_id)
);
```

Figure 5: Creating the Participant_details Table

```sql
CREATE TABLE Prize (
    prize_id INT PRIMARY KEY AUTO_INCREMENT,
    hackathon_id INT,
    Price_rank INT,
    team_id INT,
    amount DECIMAL(10,2),
    FOREIGN KEY (hackathon_id) REFERENCES Hackathon(hackathon_id),
    FOREIGN KEY (team_id) REFERENCES Team(team_id)
);
```

Figure 6: Creating the Prize Table

```sql
CREATE TABLE Submission (
    submission_id INT PRIMARY KEY AUTO_INCREMENT,
    team_id INT,
    hackathon_id INT,
    project_title VARCHAR(100),
    submission_link VARCHAR(255),
    submitted_on DATE,
    FOREIGN KEY (team_id) REFERENCES Team(team_id),
    FOREIGN KEY (hackathon_id) REFERENCES Hackathon(hackathon_id)
);
```

Figure 7: Creating the Submission Table

```sql
CREATE TABLE Team (
    team_id INT PRIMARY KEY AUTO_INCREMENT,
    team_name VARCHAR(100),
    leader_name VARCHAR(100),
    email VARCHAR(100),
    contact_number VARCHAR(15)
);
```

Figure 8: Creating the Team Table

6. Sample Queries and Outputs along with snapshots

```
mysql> select * from evaluation;
+---------------+----------+---------------+-------+----------------------+
| evaluation_id | judge_id | submission_id | score | remarks              |
+---------------+----------+---------------+-------+----------------------+
|             1 |        1 |             1 | 85.00 | Innovative project.  |
|             2 |        2 |             2 | 90.00 | Excellent idea.      |
|             3 |        3 |             3 | 88.00 | Good implementation. |
|             4 |        4 |             4 | 92.00 | User-friendly UI.    |
|             5 |        5 |             5 | 95.00 | Highly impactful.    |
+---------------+----------+---------------+-------+----------------------+
5 rows in set (0.02 sec)
```

Figure 9: Listing all Evaluations

```
mysql> select * from hackathon_details;
+--------------+------------+------------+------------+---------------------+
| hackathon_id | Fullname   | start_date | end_date   | submission_deadline |
+--------------+------------+------------+------------+---------------------+
|            1 | CodeFest   | 2025-06-01 | 2025-06-05 | 2025-06-04          |
|            2 | HackX      | 2025-07-10 | 2025-07-15 | 2025-07-14          |
|            3 | InnoHack   | 2025-08-01 | 2025-08-03 | 2025-08-02          |
|            4 | TechJam    | 2025-09-01 | 2025-09-05 | 2025-09-04          |
|            5 | HackSprint | 2025-10-10 | 2025-10-12 | 2025-10-11          |
+--------------+------------+------------+------------+---------------------+
5 rows in set (0.02 sec)
```

Figure 10: Listing all Hackathon Events

```
mysql> select * from participant_details;
+----------------+---------+------+--------------+------------+
| participant_id | team_id | name | email        | role       |
+----------------+---------+------+--------------+------------+
|              1 |       1 | P1   | p1@mail.com  | Leader     |
|              2 |       1 | P2   | p2@mail.com  | Developer  |
|              3 |       1 | P3   | p3@mail.com  | Designer   |
|              4 |       1 | P4   | p4@mail.com  | Tester     |
|              5 |       1 | P5   | p5@mail.com  | Researcher |
|              6 |       2 | P6   | p6@mail.com  | Leader     |
|              7 |       2 | P7   | p7@mail.com  | Developer  |
|              8 |       2 | P8   | p8@mail.com  | Designer   |
|              9 |       2 | P9   | p9@mail.com  | Tester     |
|             10 |       2 | P10  | p10@mail.com | Researcher |
|             11 |       3 | P11  | p11@mail.com | Leader     |
|             12 |       3 | P12  | p12@mail.com | Developer  |
|             13 |       3 | P13  | p13@mail.com | Designer   |
|             14 |       3 | P14  | p14@mail.com | Tester     |
|             15 |       3 | P15  | p15@mail.com | Researcher |
|             16 |       4 | P16  | p16@mail.com | Leader     |
|             17 |       4 | P17  | p17@mail.com | Developer  |
|             18 |       4 | P18  | p18@mail.com | Designer   |
|             19 |       4 | P19  | p19@mail.com | Tester     |
|             20 |       4 | P20  | p20@mail.com | Researcher |
|             21 |       5 | P21  | p21@mail.com | Leader     |
|             22 |       5 | P22  | p22@mail.com | Developer  |
|             23 |       5 | P23  | p23@mail.com | Designer   |
|             24 |       5 | P24  | p24@mail.com | Tester     |
|             25 |       5 | P25  | p25@mail.com | Researcher |
+----------------+---------+------+--------------+------------+
25 rows in set (0.00 sec)
```

Figure 11: Listing all Participants

```
mysql> SELECT s.submission_id, s.team_id, e.score
    -> FROM submission s
    -> JOIN evaluation e ON s.submission_id = e.submission_id
    -> WHERE e.score > 85;
+---------------+---------+-------+
| submission_id | team_id | score |
+---------------+---------+-------+
|             2 |       2 | 90.00 |
|             3 |       3 | 88.00 |
|             4 |       4 | 92.00 |
|             5 |       5 | 95.00 |
+---------------+---------+-------+
4 rows in set (0.00 sec)
```

Figure 12: Showing the submissions with Scores Above 85

```
mysql> SELECT project_title FROM Submission WHERE submitted_on > '2025-04-05';
+---------------+
| project_title |
+---------------+
| SmartBin      |
| EcoTrack      |
| EduBot        |
| SafeDrive     |
| AgroAid       |
+---------------+
5 rows in set (0.02 sec)
```

Figure 13: Showing the all submissions submitted after April 5, 2025

```
mysql> SELECT name FROM Participant_details WHERE team_id = 1;
+------+
| name |
+------+
| P1   |
| P2   |
| P3   |
| P4   |
| P5   |
+------+
5 rows in set (0.02 sec)
```

Figure 14: Showing the all participants in team 1

```
mysql> SELECT Fullname, start_date FROM Hackathon_details ORDER BY start_date;
+------------+------------+
| Fullname   | start_date |
+------------+------------+
| CodeFest   | 2025-06-01 |
| HackX      | 2025-07-10 |
| InnoHack   | 2025-08-01 |
| TechJam    | 2025-09-01 |
| HackSprint | 2025-10-10 |
+------------+------------+
5 rows in set (0.00 sec)
```

Figure 15: Showing the list hackathons ordered by start date

```
mysql> SELECT p.name, pr.team_id, pr.amount
    -> FROM participant_details p
    -> LEFT JOIN prize pr ON p.team_id = pr.team_id;
+------+---------+----------+
| name | team_id | amount   |
+------+---------+----------+
| P1   |       1 | 11000.00 |
| P2   |       1 | 11000.00 |
| P3   |       1 | 11000.00 |
| P4   |       1 | 11000.00 |
| P5   |       1 | 11000.00 |
| P6   |       2 |  8000.00 |
| P7   |       2 |  8000.00 |
| P8   |       2 |  8000.00 |
| P9   |       2 |  8000.00 |
| P10  |       2 |  8000.00 |
| P11  |       3 |  9500.00 |
| P12  |       3 |  9500.00 |
| P13  |       3 |  9500.00 |
| P14  |       3 |  9500.00 |
| P15  |       3 |  9500.00 |
| P16  |       4 |  9000.00 |
| P17  |       4 |  9000.00 |
| P18  |       4 |  9000.00 |
| P19  |       4 |  9000.00 |
| P20  |       4 |  9000.00 |
| P21  |       5 | 10000.00 |
| P22  |       5 | 10000.00 |
| P23  |       5 | 10000.00 |
| P24  |       5 | 10000.00 |
| P25  |       5 | 10000.00 |
+------+---------+----------+
25 rows in set (0.00 sec)
```

Figure 16: Showing the list All Participants with Prize Info

```
mysql> select * from prize;
+----------+--------------+------------+---------+----------+
| prize_id | hackathon_id | prize_rank | team_id | amount   |
+----------+--------------+------------+---------+----------+
|        1 |            1 |          1 |       5 | 10000.00 |
|        2 |            2 |          1 |       2 |  8000.00 |
|        3 |            3 |          1 |       4 |  9000.00 |
|        4 |            4 |          1 |       1 | 11000.00 |
|        5 |            5 |          1 |       3 |  9500.00 |
+----------+--------------+------------+---------+----------+
5 rows in set (0.00 sec)
```

Figure 17: Showing List all Prize

```
mysql> SELECT Fullname, email
    -> FROM judge
    -> WHERE expertise LIKE '%AI%';
+-----------+-----------------+
| Fullname  | email           |
+-----------+-----------------+
| Dr. Smith | smith@judge.com |
+-----------+-----------------+
1 row in set (0.02 sec)
```

Figure 18: Showing All judges with 'AI' expertise

```
mysql> SELECT project_title, team_id, submitted_on
    -> FROM submission
    -> WHERE hackathon_id = 2;
+---------------+---------+--------------+
| project_title | team_id | submitted_on |
+---------------+---------+--------------+
| EcoTrack      |       2 | 2025-07-13   |
+---------------+---------+--------------+
1 row in set (0.00 sec)
```

Figure 19: Showing List all projects submitted for Hackathon 2

```
mysql> SELECT team_id, COUNT(*) AS num_participants
    -> FROM participant_details
    -> GROUP BY team_id;
+---------+------------------+
| team_id | num_participants |
+---------+------------------+
|       1 |                5 |
|       2 |                5 |
|       3 |                5 |
|       4 |                5 |
|       5 |                5 |
+---------+------------------+
5 rows in set (0.02 sec)
```

Figure 20: Showing Count number of participants per team

```
mysql> SELECT h.Fullname AS hackathon_name, s.project_title
    -> FROM submission s
    -> JOIN hackathon_details h ON s.hackathon_id = h.hackathon_id;
+---------------+---------------+
| hackathon_name | project_title |
+---------------+---------------+
| CodeFest       | SmartBin      |
| HackX          | EcoTrack      |
| InnoHack       | EduBot        |
| TechJam        | SafeDrive     |
| HackSprint     | AgroAid       |
+---------------+---------------+
5 rows in set (0.00 sec)
```

Figure 21: Showing the hackathon and title of each submitted project

```
mysql> SELECT name, email, team_id
    -> FROM participant_details
    -> WHERE role = 'Leader';
+------+--------------+---------+
| name | email        | team_id |
+------+--------------+---------+
| P1   | p1@mail.com  |       1 |
| P6   | p6@mail.com  |       2 |
| P11  | p11@mail.com |       3 |
| P16  | p16@mail.com |       4 |
| P21  | p21@mail.com |       5 |
+------+--------------+---------+
5 rows in set (0.00 sec)
```

Figure 22: Showing the details of all participants who are 'Leaders'

```
mysql> SELECT submission_link
    -> FROM submission
    -> WHERE project_title = 'EcoTrack';
+------------------+
| submission_link  |
+------------------+
| http://link2.com |
+------------------+
1 row in set (0.00 sec)
```

Figure 23: Showing the submission link for the project titled "EcoTrack"

7. Challenges Faced

While building the Hackathon Management System, I came across several challenges that helped me to understand how to work with databases more effectively. Some of the main issues faced are listed below:

- **Designing the database structure:** It was difficult to decide how to organize the data for teams, participants, hackathons, submissions, and judges in a clear and efficient way.

- **Handling complex relationships:** Some parts of the system, like the way submissions are evaluated by multiple judges, required many-to-many relationships. We had to use extra tables to manage these properly.

- **Writing and testing SQL queries:** Making queries that work in all situations was tricky. I have to test them with different types of data to make sure they worked even in cases, like when there were no submissions or prizes.

- **Making the ER diagram easy to understand:** Creating a clear ER diagram with all entities and relationships was a bit tricky, especially when I added min-max notation and many-to-many relations.

8. Conclusion

Working on this DBMS project provided a solid understanding of how databases play a key role in real-world applications, especially in managing large-scale online systems like a virtual hackathon.

I learned how to create tables, connect them using relationships, and manage data smoothly. It gave me a clear idea of how information flows in an online system and how important it is to keep the data correct and organized.

By simulating a real-life scenario, it made learning database management systems more meaningful and helped connect theoretical knowledge with practical implementation. Overall, it was a great learning experience that showed how powerful and useful databases are.

9. Project's Learning Outcome

- Gained the ability to design an Entity-Relationship (ER) model and effectively convert it into a relational schema.
- Practiced writing SQL queries to fetch, insert, and update data.
- Understood normalization and query optimization.
- Learned how to connect tables using foreign keys and maintain relationships.
- Got better at solving real-world problems using databases.
- Understood how data flows in a system and how to manage it well.

10. References

- W3Schools. SQL Tutorial. Retrieved from `https://www.w3schools.com/sql/`
- GeeksforGeeks. SQL and DBMS Concepts. Retrieved from `https://www.geeksforgeeks.org/dbms/`
- OpenAI ChatGPT. Help with SQL and LaTeX Formatting. `https://chat.openai.com`
- Youtube video- Help in installation of MySQL Workbench `https://youtube.com/shorts/zk5_Eikr90E?si=GbwWLYvpYEeim_m9`
- Python Book for reference `https://www.ssgopalganj.in/online/Class%20XII/Comp/Computer%20science%20PYTHON%20book%20pdf%20for%20class%2012.pdf`
- MySQL Documentation - `https://dev.mysql.com/doc/`

Appendix: SQL Code

This appendix includes all the SQL code used in the Virtual Hackathon Tracker project. Each part of the database is designed with a clear purpose, ensuring data is well-organized and easy to retrieve. Below is the complete SQL used to create tables, define relationships, and perform key queries. Comments are added to explain the purpose of each line of code.

1. Table Creation Scripts

```
-- TEAM table stores details of all the teams registered in the hackathon.

CREATE TABLE Team (
    team_id INT PRIMARY KEY AUTO_INCREMENT,  -- Unique identifier for each team
    team_name VARCHAR(100),                 -- Name of the team
    leader_name VARCHAR(100),               -- Name of the team leader
    email VARCHAR(100),                     -- Contact email of the team leader
    contact_number VARCHAR(15)              -- Phone number for communication
);

--PARTICIPANT_DETAILS table stores individual participant data and their team
association.

CREATE TABLE Participant_details (
    participant_id INT PRIMARY KEY AUTO_INCREMENT, -- Unique identifier for
                                                      each participant
    team_id INT,                                  -- Foreign key that links to
                                                     the Team table
    name VARCHAR(100),                         -- Full name of the participant
    email VARCHAR(100),                        -- Email ID of the participant
    role VARCHAR(50),      - Role in the project (e.g., Developer, Designer)
    FOREIGN KEY (team_id) REFERENCES Team(team_id) -- Maintains team
    relationship
);
```

```
-- HACKATHON_DETAILS table contains event-specific data about each hackathon.

CREATE TABLE Hackathon_details (
    hackathon_id INT PRIMARY KEY AUTO_INCREMENT, --Unique ID for hackathon
    Fullname VARCHAR(100),                -- Full name/title of the hackathon
    start_date DATE,                      -- Start date of the hackathon
    end_date DATE,                        -- End date of the hackathon
    submission_deadline DATE              -- Last date to submit projects
);

-- SUBMISSION table holds the projects submitted by teams for a specific
   hackathon.

CREATE TABLE Submission (
    submission_id INT PRIMARY KEY AUTO_INCREMENT,  -- Unique submission ID
    team_id INT,           -- Foreign key to identify which team submitted
    hackathon_id INT,      -- Foreign key to identify which hackathon it's for
    project_title VARCHAR(100),         -- Title of the project
    submission_link VARCHAR(255),       -- Link to the GitHub repo, demo, etc.
    submitted_on DATE,                  -- Date when the project was submitted
    FOREIGN KEY (team_id) REFERENCES Team(team_id),
    FOREIGN KEY (hackathon_id) REFERENCES Hackathon_details(hackathon_id)
);

-- JUDGE table keeps details about the judges evaluating the projects.

CREATE TABLE Judge (
    judge_id INT PRIMARY KEY AUTO_INCREMENT, -- Unique ID for each judge
    Fullname VARCHAR(100),                   -- Full name of the judge
    email VARCHAR(100),                      -- Contact email
    expertise VARCHAR(100)                   -- Area of expertise (EX-AI,dev.)
);




-- EVALUATION table stores scores and remarks given by judges for each
```

```
   submission.


CREATE TABLE Evaluation (
    evaluation_id INT PRIMARY KEY AUTO_INCREMENT,-- Unique evaluation recordID
    judge_id INT,                -- Judge who evaluated the project
    submission_id INT,           -- The submission being evaluated
    score D                      -- Score given to the project (e.g., 87.50)
    remarks VARCHAR(255),        -- Feedback or comments by the judge
    FOREIGN KEY (judge_id) REFERENCES Judge(judge_id),
    FOREIGN KEY (submission_id) REFERENCES Submission(submission_id)
);

-- PRIZE table stores results of the hackathon including prize money and rank.

CREATE TABLE Prize (
    prize_id INT PRIMARY KEY AUTO_INCREMENT,       -- Unique ID for each
                                                      prize record

    hackathon_id INT,     -- Which hackathon the prize was awarded in
    rank_ INT,                                     -- Position secured by the team (e
    team_id INT,                         -- Which team won the prize
    amount DECIMAL(10,2),                -- Amount of prize money awarded
    FOREIGN KEY (hackathon_id) REFERENCES Hackathon_details(hackathon_id),
    FOREIGN KEY (team_id) REFERENCES Team(team_id)
);
```

2. Sample SQL Queries with Explanations

```
mysql> select * from evaluation;
+---------------+----------+---------------+-------+----------------------+
| evaluation_id | judge_id | submission_id | score | remarks              |
+---------------+----------+---------------+-------+----------------------+
|             1 |        1 |             1 | 85.00 | Innovative project.  |
|             2 |        2 |             2 | 90.00 | Excellent idea.      |
|             3 |        3 |             3 | 88.00 | Good implementation. |
|             4 |        4 |             4 | 92.00 | User-friendly UI.    |
|             5 |        5 |             5 | 95.00 | Highly impactful.    |
+---------------+----------+---------------+-------+----------------------+
5 rows in set (0.02 sec)
```

Figure 24: Listing all Evaluations

`Explaination: Showing all the records of Evaluation table.`

```
mysql> select * from prize;
+----------+--------------+------------+---------+----------+
| prize_id | hackathon_id | prize_rank | team_id | amount   |
+----------+--------------+------------+---------+----------+
|        1 |            1 |          1 |       5 | 10000.00 |
|        2 |            2 |          1 |       2 |  8000.00 |
|        3 |            3 |          1 |       4 |  9000.00 |
|        4 |            4 |          1 |       1 | 11000.00 |
|        5 |            5 |          1 |       3 |  9500.00 |
+----------+--------------+------------+---------+----------+
5 rows in set (0.00 sec)
```

Figure 25: Showing List all Prize

`Explaination: Showing all the records of Prize  table.`

```
mysql> SELECT name FROM Participant_details WHERE team_id = 1;
+------+
| name |
+------+
| P1   |
| P2   |
| P3   |
| P4   |
| P5   |
+------+
5 rows in set (0.02 sec)
```

Figure 26: Showing the all participants in team 1

`Explaination: Showing all the records of Participant  table where team is 1.`

```
mysql> select * from participant_details;
+----------------+---------+------+--------------+------------+
| participant_id | team_id | name | email        | role       |
+----------------+---------+------+--------------+------------+
|              1 |       1 | P1   | p1@mail.com  | Leader     |
|              2 |       1 | P2   | p2@mail.com  | Developer  |
|              3 |       1 | P3   | p3@mail.com  | Designer   |
|              4 |       1 | P4   | p4@mail.com  | Tester     |
|              5 |       1 | P5   | p5@mail.com  | Researcher |
|              6 |       2 | P6   | p6@mail.com  | Leader     |
|              7 |       2 | P7   | p7@mail.com  | Developer  |
|              8 |       2 | P8   | p8@mail.com  | Designer   |
|              9 |       2 | P9   | p9@mail.com  | Tester     |
|             10 |       2 | P10  | p10@mail.com | Researcher |
|             11 |       3 | P11  | p11@mail.com | Leader     |
|             12 |       3 | P12  | p12@mail.com | Developer  |
|             13 |       3 | P13  | p13@mail.com | Designer   |
|             14 |       3 | P14  | p14@mail.com | Tester     |
|             15 |       3 | P15  | p15@mail.com | Researcher |
|             16 |       4 | P16  | p16@mail.com | Leader     |
|             17 |       4 | P17  | p17@mail.com | Developer  |
|             18 |       4 | P18  | p18@mail.com | Designer   |
|             19 |       4 | P19  | p19@mail.com | Tester     |
|             20 |       4 | P20  | p20@mail.com | Researcher |
|             21 |       5 | P21  | p21@mail.com | Leader     |
|             22 |       5 | P22  | p22@mail.com | Developer  |
|             23 |       5 | P23  | p23@mail.com | Designer   |
|             24 |       5 | P24  | p24@mail.com | Tester     |
|             25 |       5 | P25  | p25@mail.com | Researcher |
+----------------+---------+------+--------------+------------+
25 rows in set (0.00 sec)
```

Figure 27: Listing all Participants

Explaination: Showing all the records of Participants table.

```
mysql> INSERT INTO Participant_details (team_id, name, email, role)
    -> VALUES
    -> (6, 'P26', 'p26@mail.com', 'Backend Developer'),
    -> (6, 'P27', 'p27@mail.com', 'UI/UX Designer'),
    -> (6, 'P28', 'p28@mail.com', 'DevOps Engineer'),
    -> (7, 'P29', 'p29@mail.com', 'Frontend Developer');
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

Figure 28: Inserting records in the Participants table

Explaination: Inserting some new records in the Participant table.

```
mysql> INSERT INTO Hackathon_details (Fullname, start_date, end_date, submission_deadline)
    -> VALUES
    -> ('Hack-a-thon 2025', '2025-04-01', '2025-04-05', '2025-04-04'),
    -> ('CodeFest Global', '2025-05-10', '2025-05-12', '2025-05-11');
Query OK, 2 rows affected (0.03 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

Figure 29: Inserting records in the hackathon table

Explaination: Inserting some new records in the Hackathon_details table.

```
mysql> INSERT INTO Team (team_name, leader_name, email, contact_number)
    -> VALUES
    -> ('Code Warriors', 'P1', 'p1@mail.com', '9991110001'),
    -> ('HackSmiths', 'P6', 'p6@mail.com', '9991110002'),
    -> ('Bit Busters', 'P11', 'p11@mail.com', '9991110003'),
    -> ('Script Ninjas', 'P16', 'p16@mail.com', '9991110004'),
    -> ('Byte Force', 'P21', 'p21@mail.com', '9991110005');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Figure 30: Inserting records in the team table

Explaination: Inserting some new records in the Team table.

```
mysql> INSERT INTO Prize (hackathon_id, prize_rank, team_id, amount)
    -> VALUES
    -> (1, 1, 3, 5000.00),
    -> (1, 2, 1, 3000.00),
    -> (1, 3, 2, 2000.00),
    -> (2, 1, 5, 6000.00),
    -> (2, 2, 4, 3500.00);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Figure 31: Inserting records in the Prize table

`Explaination: Inserting some new records in the Prize table.`

```
mysql> SELECT Fullname, email
    -> FROM judge
    -> WHERE expertise LIKE '%AI%';
+-----------+-----------------+
| Fullname  | email           |
+-----------+-----------------+
| Dr. Smith | smith@judge.com |
+-----------+-----------------+
1 row in set (0.02 sec)
```

Figure 32: Showing All judges with 'AI' expertise

`Explaination: Showing all the records of judges who are expert in AI.`

```
mysql> SELECT s.submission_id, s.team_id, e.score
    -> FROM submission s
    -> JOIN evaluation e ON s.submission_id = e.submission_id
    -> WHERE e.score > 85;
+---------------+---------+-------+
| submission_id | team_id | score |
+---------------+---------+-------+
|             2 |       2 | 90.00 |
|             3 |       3 | 88.00 |
|             4 |       4 | 92.00 |
|             5 |       5 | 95.00 |
+---------------+---------+-------+
4 rows in set (0.00 sec)
```

Figure 33: Showing the submissions with Scores Above 85

`Explaination: Showing all the records of submissions with scores above 85`

```
mysql> SELECT s.project_title, e.score
    -> FROM Evaluation e
    -> JOIN Submission s ON e.submission_id = s.submission_id
    -> ORDER BY e.score DESC
    -> LIMIT 3;
+---------------+-------+
| project_title | score |
+---------------+-------+
| AgroAid       | 95.00 |
| SafeDrive     | 92.00 |
| EcoTrack      | 90.00 |
+---------------+-------+
3 rows in set (0.02 sec)
```

Figure 34: Top 3 Submissions by Score

`Explanation: Retrieves the top 3 scoring projects from all evaluations.`

```
mysql> SELECT team_id, COUNT(*) AS total_participants
    -> FROM Participant_details
    -> GROUP BY team_id;
+---------+--------------------+
| team_id | total_participants |
+---------+--------------------+
|       1 |                  5 |
|       2 |                  5 |
|       3 |                  5 |
|       4 |                  5 |
|       5 |                  5 |
|       6 |                  3 |
|       7 |                  1 |
+---------+--------------------+
7 rows in set (0.01 sec)
```

Figure 35: Number of Participants Per Team

Explanation: Uses COUNT and GROUP BY to calculate how many participants are in each team.

```
mysql> SELECT name, role
    -> FROM Participant_details
    -> WHERE team_id = 1;
+------+------------+
| name | role       |
+------+------------+
| P1   | Leader     |
| P2   | Developer  |
| P3   | Designer   |
| P4   | Tester     |
| P5   | Researcher |
+------+------------+
5 rows in set (0.02 sec)
```

Figure 36: Teams Without Submission

Explanation: Finds teams that haven't submitted any project using LEFT JOIN and NULL check.

```
mysql> SELECT DISTINCT h.hackathon_id, h.Fullname
    -> FROM Evaluation e
    -> JOIN Submission s ON e.submission_id = s.submission_id
    -> JOIN Hackathon_details h ON s.hackathon_id = h.hackathon_id
    -> WHERE e.judge_id = 1;
+--------------+----------+
| hackathon_id | Fullname |
+--------------+----------+
|            1 | CodeFest |
+--------------+----------+
1 row in set (0.02 sec)
```

Figure 37: Hackathons Evaluated by Judge ID 1

`Explanation: Shows unique hackathons where Judge 1 has evaluated submissions.`

3. Notes on SQL Structure and Assumptions

- All primary keys are auto-incremented for uniqueness and easy tracking.
- Foreign keys help enforce referential integrity between tables.
- VARCHAR types were chosen for names, emails, and remarks for flexibility.
- Relationships like team-submission, judge-evaluation, and hackathon-prize are one-to-many.
- Dates are stored using the `DATE` type to allow easy filtering and sorting.
- This schema assumes each submission is linked to one hackathon and one team.
- A team can make only one submission per hackathon in this version.

Appendix: ER Diagram

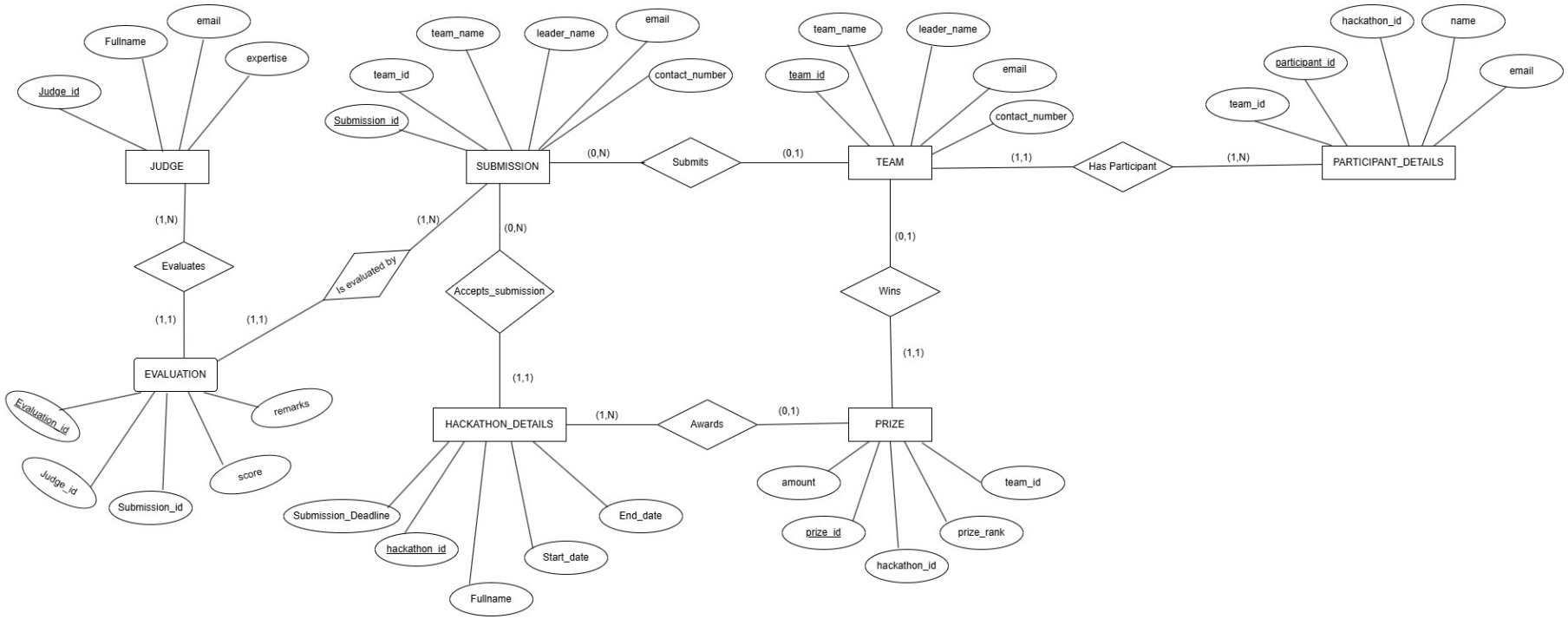

Figure 38: Virtual Hackathon Tracker